

AI 연산 가속기 최적화 고효율 병렬 스토리지 SW 기술 개발

거대모델 기반 데이터 파이프라인의 가속을 위한 오케스트레이션 기술 개발

2차년도 2차 워크숍

2025.11.06 (목)

한국전자기술연구원

책임 안재훈

- 주관기관 : (주)글루시스
- 공동연구 기관 : 한국전자통신연구원(ETRI)
한국전자기술연구원(KETI)
한국정보통신기술협회(TTA)
한국컴퓨팅산업협회
성균관대학교
포항공과대학교

목차

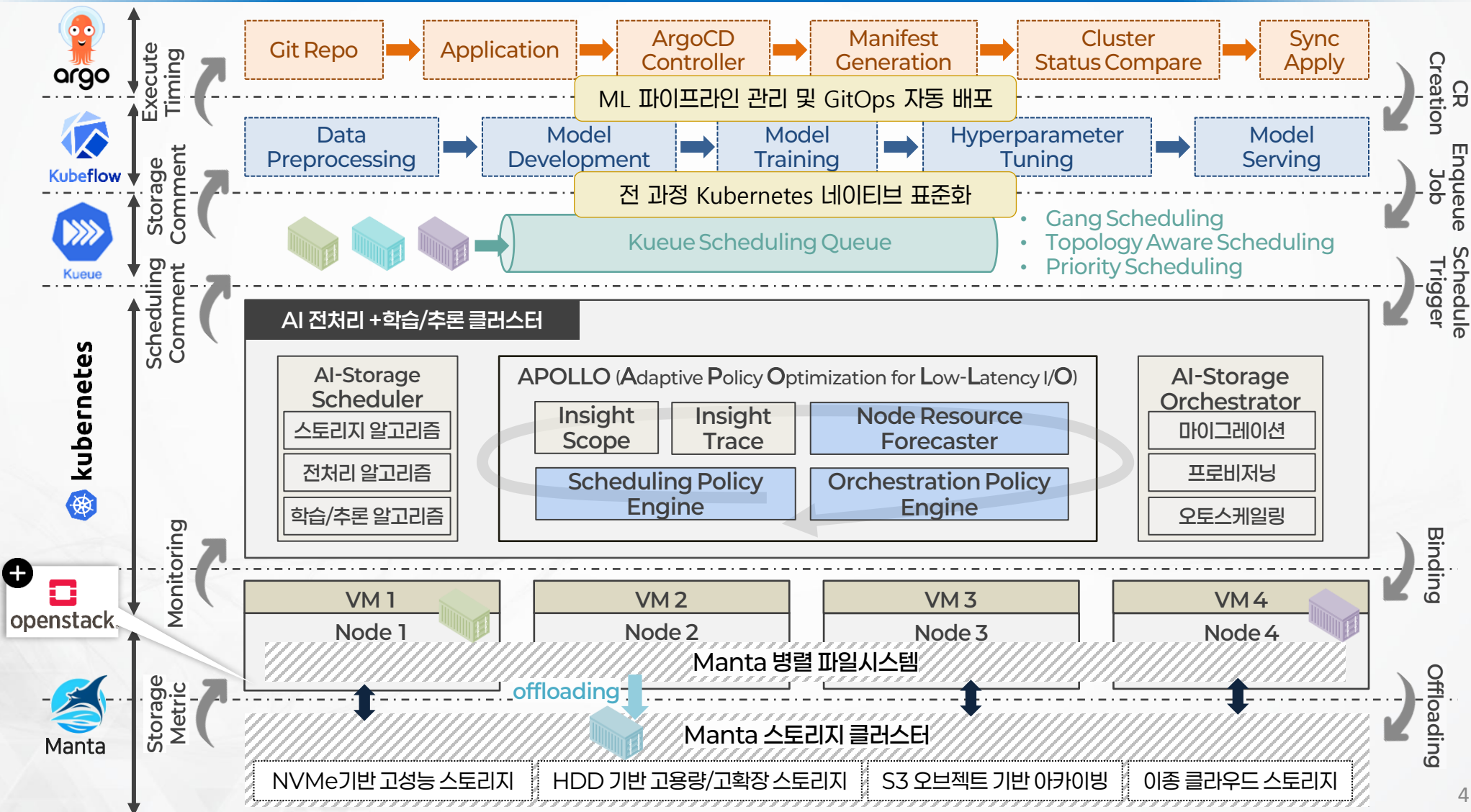
- I. 2차년도 개발 진행 사항
- II. 3차년도 진행 예정 사항
- III. 2차년도 산출물 및 향후 일정

CHAPTER

2차년도 개발 진행 사항

1. KETI 개발 아키텍처

기존 MLOps 파이프라인과 결합 가능한 KETI 지능형 오케스트레이션



2. MLOps 파이프라인 결합형 확장 오케스트레이션

개발 목표

Kubeflow·ArgoCD·Kueue 인지형 오케스트레이션 개발

개발 내용

- ArgoCD Git 기반 Kubeflow 리소스 자동 배포 및 APOLLO 정책 연동
- Kueue Scheduling Queue 기반 우선순위 스케줄링과 KETI Scheduler 노드 배치 통합

↓ (Pod 생성 흐름)

1 Kubeflow Custom Resource 생성

```
git commit -m "add TFJob for training"
```

```
pytorchjobs.kubeflow.org
tfjobs.kubeflow.org
```

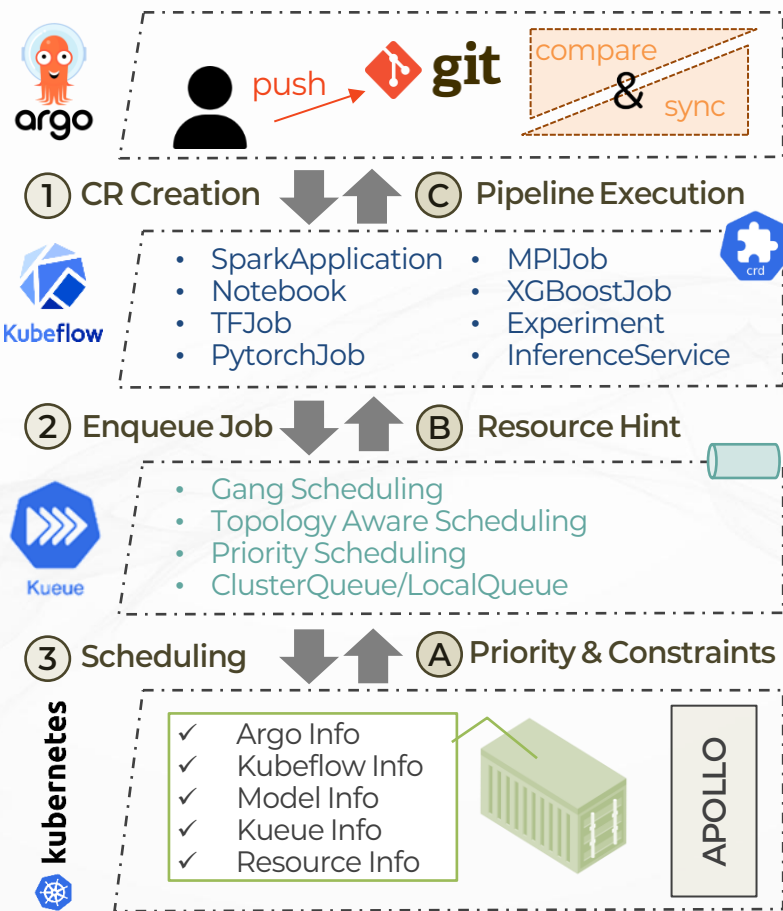
2 Kueue의 CR Job 대기열 추가

```
kubectl get localqueues -A
```

NAME	COHORT	PENDING	ADMITTED
gpu-queue	ai	3	3

3 K8s Pod 형태로 KETI 오케스트레이션

```
apiVersion: kubeflow.org/v1
kind: TFJob
metadata:
  name: pytorch-simple
  namespace: default
  labels:
    + kueue.x-k8s.io/queue-name: gpu-queue
spec:
  ...
  template:
    spec:
      + schedulerName: keti-scheduler
```



↑ (피드백 흐름)

C 파이프라인 실행 타이밍 및 정기 작업 상태

```
metadata:
  annotations:
    keti.apollo/duration: "15min"
    keti.apollo/storage-ready: "ready"
    keti.apollo/recommend: "immediate"
    keti.apollo/bottleneck: "none"
```

B 재시도 시 사용할 Resource Hints

```
Run:
  name: train-job-002
  properties:
    kueue.queueWaitSeconds: "183"
    kueue.chosenFlavor: "gpu-a100x1"
    kueue.recommendation: "mem:+2Gi"
```

A 스케줄링 우선 순위 및 노드 배치 제약 조건

```
metadata:
  labels:
    keti.apollo/urgency: "high"
  annotations:
    keti.apollo/priority: "10"
    keti.apollo/gang_size: "4"
```


3. AI 워크로드 메타데이터 처리 특성 분석

개발 목표

Pod Manifest(yaml) 기반 **AI 워크로드**(전처리/학습/추론) **특징 추출**

개발 내용

- 전처리 특화 시그니처(데이터 타입, 증강 유형, 변환 연산) 추출 및 I/O 패턴 분류 기술
- 이미지/텍스트/오디오별 전처리 작업의 읽기/쓰기 비율 및 스토리지 계층 최적 매칭

[관련 SW]

- Insight Scope



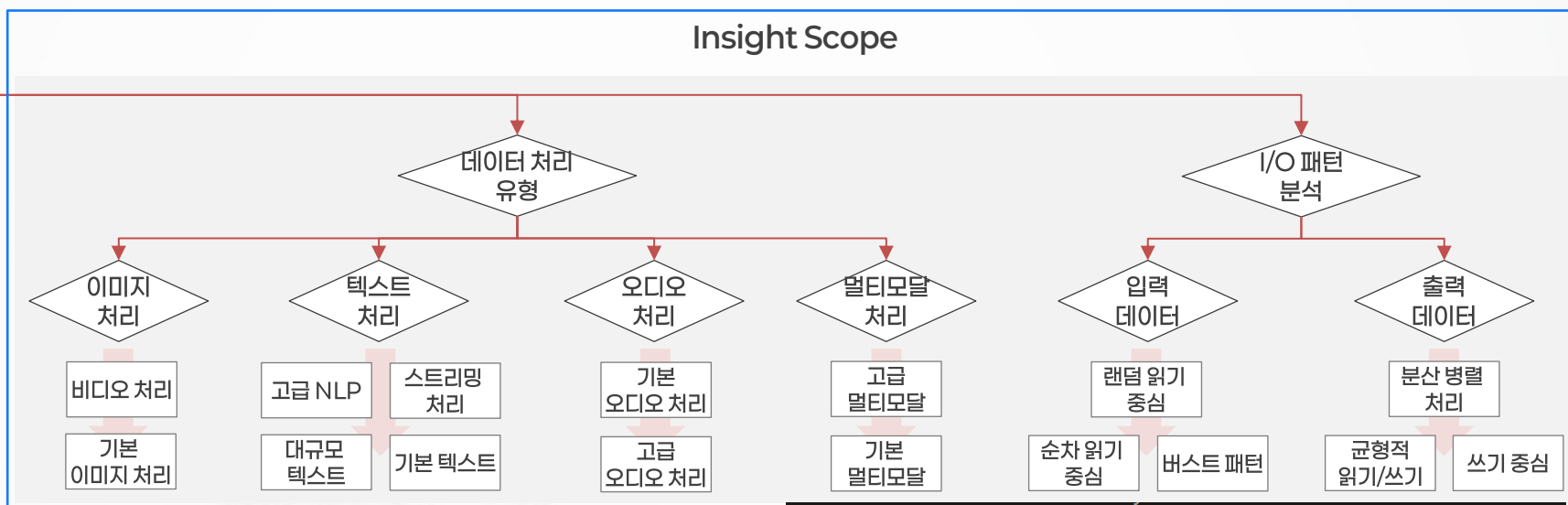
AI 워크로드

Image
Volume
Args
Resource

시그니처 추출

- ✓ 데이터 타입
- ✓ I/O 패턴
- ✓ 스토리지 티어

Insight Scope



Container Type: Spark-based image preprocessing
I/O Pattern: (1) High throughput sequential read (2) moderate random write
Storage Requirement: NVMe-based

❖ Pod yaml 기반 전처리 특화 시그니처 분석

yaml 요소	추출 정보 예시	활용 목적
컨테이너 이미지 패턴	apache / spark : 3.4	✓ 프레임워크 종류로 기본 I/O 특성 추론
명령어 키워드	args[] : --data-type = image	✓ 데이터 타입과 배치 크기로 I/O 패턴 및 처리량 예측
볼륨 마운트 분석	volumeMounts[] . mountPath = "/data/raw"	✓ 읽기/쓰기 비율과 데이터 크기로 스토리지 및 부하 산정
자원 요청 분석	resources. requests.cpu : "4"	✓ CPU/GPU/메모리 요구량으로 I/O 병목 가능성 파악
라벨 / 어노테이션	metadata. labels ["data-type"] = "image"	✓ 사용자 명시적 힌트로 추론 정확도 보완

Dataset Info

DAG
Dependency

Execution
History



Argo 기반 추가 정보

4. 스케줄링 정책 결정 및 자동 최적화 엔진

개발 목표

강화학습 기반 스케줄링 정책 최적화 모델

개발 내용

- 워크로드 시그니처 및 모니터링 기반 최적 스케줄링 정책 결정 모델 (MLOps 파이프라인 및 스토리지 I/O 고려)
- 강화학습 (PPO) 기반 목적함수 정의 및 시뮬레이션 기반 검증 완료

[관련 SW]

- Scheduling Policy Engine

Insight Scope

워크로드 시그니처

- ✓ 워크로드 타입
- ✓ 연산 복잡도
- ✓ I/O 패턴
- ✓ 데이터/연산 타입

MLOps Data

Kubeflow Info

- ✓ Job 병렬도
- ✓ 재시도 정책
- ✓ Replica 스펙

Kueue Info

- ✓ 큐 개수/상태
- ✓ Affinity
- ✓ Topology

Monitoring Data

클러스터 상태

- ✓ 노드 타입
- ✓ 분산 파일 시스템
- ✓ 스토리지 티어
- ✓ 지역성

모델 내부 목적함수 정의 및
시뮬레이션 기반 검증

Scheduling Policy Engine

Feature Extract & Embedding Layer

Workload
Signature Encoder

Pipeline Graph
Encoder

Cluster Status
Encoder

Feature Fusion
Network

임베딩 벡터 Input

RL Model Layer (Proximal Policy Optimization Based)

Critic Network

현재 상태 기반
정책의 가치 추정

$$A(s,a)=r+\gamma V(s')-V(s)$$

Actor Network

스토리지 인식
스케줄링 정책
확률 분포 생성

I/O Aware : 67%
Data-Type : 23%
Pipeline : 10%

Advantage Predictor

PPO Policy Clipping

Combined Loss Function

Loss Calculator & Optimizer

Policy Generate & Output Layer

스케줄링 정책
및 파라미터 전달

Policy Generator

Scheduling
Algorithm Selector

Scheduling
Parameter Optimizer

Format & Schema
Validator

Scheduling Policy

→ 모델 업데이트 과정

→ 스케줄링 정책 결정 과정

성능 지표 기반
가중치 업데이트

5. APOLLO 정책 엔진 기반 전처리·파이프라인 특화 스케줄링

개발 목표

RL 파라미터 기반 **스케줄링 알고리즘** 및 **MLOps 도구 연동 자동화**

[관련 SW]

- AI-Storage Scheduler

개발 내용

- 전처리 (Locality/Tier/Pattern) 및 파이프라인 (Stage/Gang/Fragment/Phase) 특화 알고리즘 7종
- 스케줄링 결과 KubeFlow Manifest 및 Kueue Admission 자동 주입

Scheduling Policy

Data Locality

- Locality_Weight
- Distance_Penalty
- Latency_ms

Storage Tier

- Tier_Match
- Io_load_penalty
- Max_io_util

I/O Pattern

- Locality_Weight
- Distance_Penalty
- Latency_ms

Pipeline Stage

- Colocation_Pref
- Reserve_Timeout
- Next_Stage_Hour

Gang

- Gang_Reservation
- Max_Latency_ms
- Retry_Attempts

Resource Fragment

- Fragment_Penalty
- Colocation_Prefer
- Future_Resource

Dynamic Phase

- Prefill_Multiplier
- Transition_Thres
- Release_Ratio

AI-Storage Scheduler

알고리즘 개수 추가 진행중

K8s Algorithm (v1.32 기준)	Default Plugin	13종	20종
	Volume 관련 Plugin	7종	
KETI 전처리 특화 Algorithm	Data Locality Aware	데이터 노드 우선 배치, 네트워크 전송 최소화	3종
	Storage Tier Aware	I/O 패턴 고려, NVMe / SSD / HDD	
	I/O Pattern Based	증강 / 변환 / 필터링 유형	
KETI ML 파이프라인 특화 Algorithm 	Pipeline Stage Aware	파이프라인 단계 고려, Colocation 전송 최소화	4종
	Gang Scheduling	분산 학습 워커 동시, All-or-Nothing	
	Resource Fragment	파이프라인 전체 리소스, 미래 리소스 파편화 고려	
	Dynamic Phase	Prefill / Decode 단계, 단계 전환 시 동적 조정	

6. 전처리 특화 알고리즘 노드 선정 프로세스

개발 목표

그래프 구조화 → 비용 계산 → 노드 선택 3단계 프로세스

[관련 SW]

- AI-Storage Scheduler

개발 내용

- 노드 그래프 구성(Locality, Distance, I/O Load) 및 RL 파라미터 적용 Edge Cost 계산
- 데이터-노드 간 그래프 최단 경로 탐색으로 전처리 네트워크 전송 및 I/O 병목 최소화

전처리 특화 스케줄링 알고리즘

Data Locality 전처리 특화 알고리즘 수행 로그

```
INFO =====
INFO Preprocessing Scheduling: Data-Locality-Aware
INFO =====
INFO Building Graph...
DEBUG Nodes: [imagenet-data, node-1, node-2, node-3]
DEBUG Edges:
DEBUG imagenet-data → node-1: base_cost=100 (Remote)
DEBUG imagenet-data → node-2: base_cost=0 (Local)
DEBUG imagenet-data → node-3: base_cost=200 (Remote)
INFO Applying RL Parameters...
DEBUG locality_weight: 0.85
DEBUG distance_penalty_factor: 1.5
INFO Calculating Edge Costs...
DEBUG imagenet-data → node-1:
DEBUG Base: 100, Penalty: 1.5x, I/O Load: 0.3
DEBUG Final Cost: 174 (100*1.5 + 0.3*80)
DEBUG imagenet-data → node-2:
DEBUG Base: 0, Locality: 0.85x, I/O Load: 0.5
DEBUG Final Cost: 40 (0*0.85 + 0.5*80)
DEBUG imagenet-data → node-3:
DEBUG Base: 200, Penalty: 1.5x, I/O Load: 0.1
DEBUG Final Cost: 308 (200*1.5 + 0.1*80)
INFO Running Dijkstra Shortest Path...
INFO ✓ Best Path Found: imagenet-data → node-2
INFO Total Cost: 40
INFO Reason: Local data access
INFO ✓ Node Selected: node-2
INFO Expected Benefits:
INFO - Network Transfer Saved: 1.2TB
INFO - Estimated Time Saved: 180s
INFO =====
```

노드 그래프 구축

스케줄링 비용 계산

노드 선정 및 배치

그래프
구조화

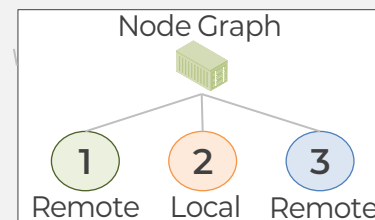
비용 계산

바인딩

Graph Based 노드 스케줄링

Data-processing

- ✓ "Locality_Weight": 0.85
- ✓ "Distance_Penalty": 1.5
- ✓ "I/O Load": 80



	Node1	Node2	Node3	
Locality	0	100	0	X 0.85
Distance	100	0	200	X 1.5
I/O Load	0.3	0.5	0.1	X 80
Final Cost	175	125	308	



Node2 : Local Data

최종 스케줄링 결정

- 전처리 스케줄링 핵심 문제 : “어디에 배치할까?”
그래프 최단 경로 문제로 모델링

7. ML 파이프라인 특화 단계 간 연결 스케줄링 알고리즘

개발 목표

파이프라인 단계 연결성 기반 **Two-Phase Commit 스케줄링**

[관련 SW]

- AI-Storage Scheduler

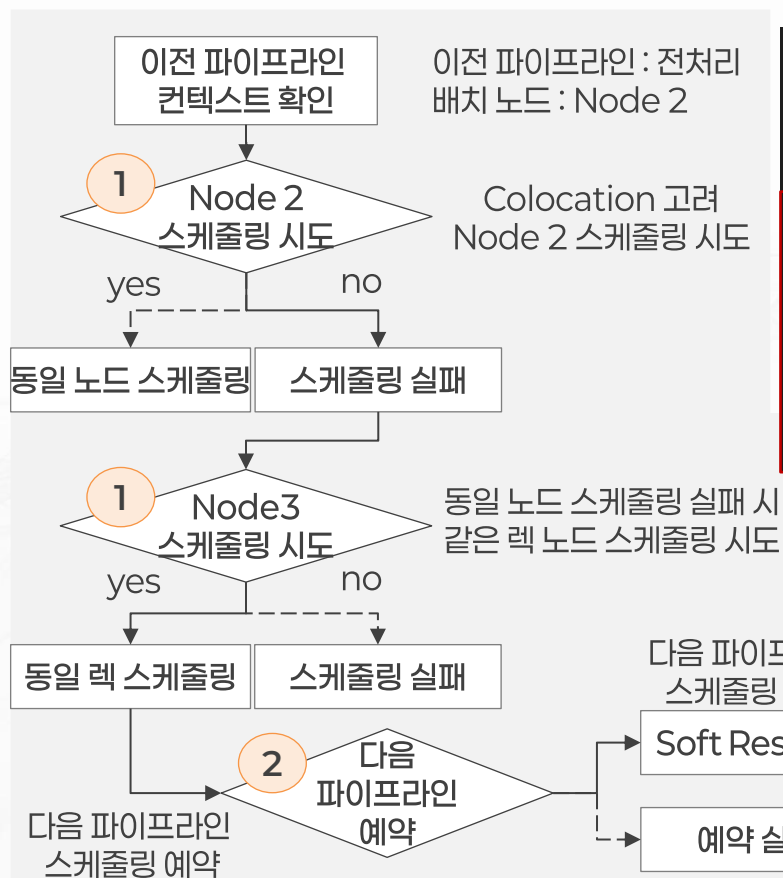
개발 내용

- Phase 1: 이전 파이프라인 컨텍스트 확인 → 동일 노드 시도 → 동일 랙 Fallback → 예약 완료
- Phase 2: 예약 확정 및 향후 Soft-Reserve 생성 후 Kubeflow/Kueue에 배치 정보 자동 전달

Pipeline-aware 알고리즘 수행 로그

```
INFO [Attempt 1] Checking previous stage node: node-2
DEBUG Node-2 Current Status:
DEBUG Total GPU: 2, Available: 1
DEBUG Total Memory: 64Gi, Available: 32Gi
DEBUG Training Requirements:
DEBUG GPU: 2, Memory: 48Gi
WARN X Insufficient resources on node-2
WARN Need: GPU=2, Available: GPU=1
INFO
INFO [Attempt 2] Fallback to same-rack nodes...
INFO Discovering rack topology...
DEBUG node-2 in rack: rack-A
DEBUG Same-rack nodes: [node-2, node-3, node-5, node-6]
DEBUG Network latency within rack-A: 0.5ms (IB)
INFO
INFO Evaluating node-3 (rack-A)...
DEBUG Available: GPU=2, Memory=64Gi ✓
DEBUG Network to node-2: 0.5ms, 100Gbps
INFO Attempting reservation on node-3...
DEBUG Timeout: 60s
INFO ✓ Reserved: node-3 (reservation-id: res-456)
INFO
INFO ===== PHASE 2: COMMIT & FUTURE RESERVE =====
INFO Committing training stage to node-3...
INFO ✓ Committed successfully
INFO Pod Name: ml-pipeline-789-training-master-0
DEBUG Resources Allocated:
DEBUG GPU: 2x NVIDIA RTX 6000 Pro
DEBUG Memory: 48Gi
DEBUG CPU: 16 cores
INFO
INFO Creating future stage reservations...
DEBUG Next Stage: evaluation (estimated start: +2 hours)
DEBUG Evaluation Resources: GPU=1, Memory=16Gi
INFO Soft-Reserve [evaluation] on node-3:
DEBUG Resources: GPU=1, Memory=16Gi
DEBUG Valid Until: 2025-11-06 13:00:00 (2.5 hours)
DEBUG Priority: medium (can yield to urgent workloads)
INFO ✓ Soft-Reserved: evaluation (soft-res-id: soft-789)
INFO
INFO Phase 2 Summary:
INFO ✓ training: Committed to node-3 (same rack as preproces
INFO ✓ evaluation: Soft-reserved on node-3 (2.5h window)
```

ML 파이프라인 특화 알고리즘



```
INFO
INFO ===== PIPELINE PLACEMENT SUMMARY =====
INFO
INFO Pipeline Flow:
INFO preprocessing → node-2 (completed)
INFO training → node-3 (running) ← Current
INFO evaluation → node-3 (reserved)
INFO
INFO Updating Kubeflow Manifest...
DEBUG Setting nodeSelector: node-3
DEBUG Adding podAffinity: pipeline-id=ml-pipeline-789
DEBUG Setting topology: rack-A
INFO
INFO Updating Kueue Workload...
DEBUG Setting admission.clusterQueue: training-queue
DEBUG Setting admission.podSetAssignments.nodeSelector: node-3
DEBUG Setting reservedResources: gpu=1, memory=16Gi (for evaluation)
DEBUG Setting reservationExpiry: 2025-11-06T13:00:00Z
INFO
INFO ✓ Pipeline-Stage-Aware Scheduling Complete
```

Kubeflow 메니페스트 Annotation

- ✓ Node Select
- ✓ Pod Affinity
- ✓ Set Topology

Kueue 스케줄링 Admission

- ✓ Node Assign
- ✓ Resource Reserved
- ✓ Expiry Time

8. ML 파이프라인 단계별 자원 사용 패턴 추적

개발 목표

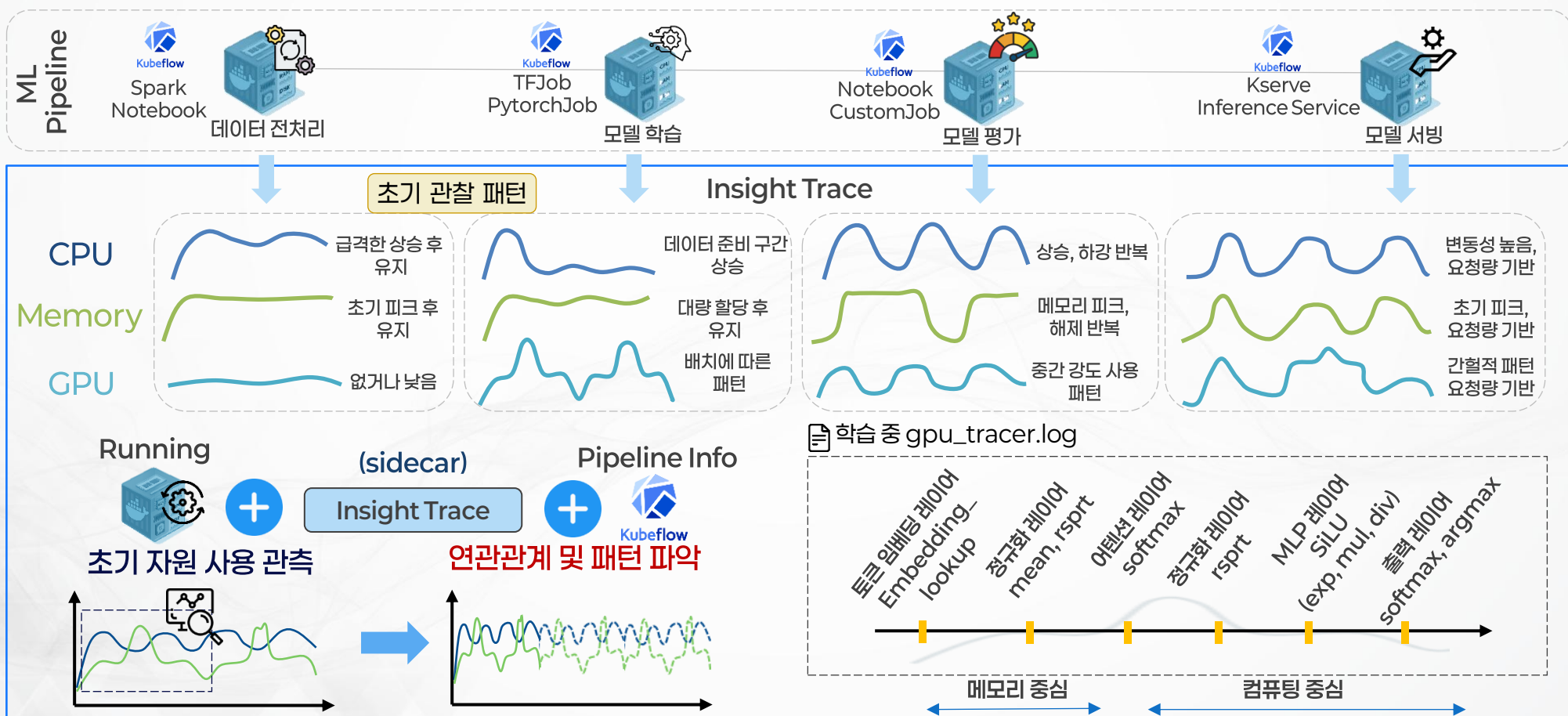
ML 파이프라인 각 단계별 사용 패턴 실시간 수집 및 연관관계 파악

개발 내용

- Insight Trace (Sidecar) 기반 파이프라인 단계별 자원 사용 패턴 실시간 모니터링
- 초기 관찰 데이터 기반 자원 간 연관관계 파악

[관련 SW]

- Insight Trace



9. 자원 사용량 예측 및 오케스트레이션 정책 결정 모델

개발 목표

2-Stage ML 기반 자원 예측 및 오케스트레이션 정책 결정

개발 내용

- 시계열 예측 모델(LSTM-Attention)기반 노드별 시계열 데이터 및 집계 정보 분석
- 시계열 데이터 및 MLOps 파이프라인 정보 기반 오케스트레이션 정책을 위한 다중 분류 모델(LightGBM)

[관련 SW]

- Resource Forecaster

Alibaba Cluster Trace

- ✓ cpu_util
- ✓ memory_util
- ✓ net_in
- ✓ net_out

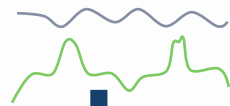
KETI 추가 근사 메트릭

- ✓ available_cores
- ✓ load_average
- ✓ workload_phase
- ✓ high_priority
- ✓ total_cpu_req
- ✓ total_mem_req

3차년도 추가 연계 메트릭

- ✓ total_iops
- ✓ throughput
- ✓ cache_hit_ratio
- ✓ io_queue_depth
- ✓ read_write_ratio
- ✓ avg_io_size
- ✓ gpu_util
- ✓ gpu_mem_util

Insight Trace

노드 내
Pod 자원 사용 패턴

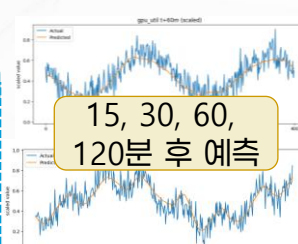
Node Resource Forecatster LSTM-Attention

Layer (type)	Output Shape	Param #	Connected to
input_layer_6 (InputLayer)	(None, 60, 12)	0	-
lstm_6 (LSTM)	(None, 60, 96)	41,856	input_layer_6[0]...
attention_6 (Attention)	(None, 60, 96)	0	lstm_6[0][0], lstm_6[0][0]
concatenate_4 (Concatenate)	(None, 60, 192)	0	lstm_6[0][0], attention_6[0][0]
global_average_poo... (GlobalAveragePool...)	(None, 192)	0	concatenate_4[0]...
dense_12 (Dense)	(None, 96)	18,528	global_average_p...
dropout_4 (Dropout)	(None, 96)	0	dense_12[0][0]
dense_13 (Dense)	(None, 16)	1,552	dropout_4[0][0]

정확도 90.3%

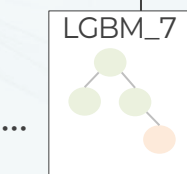
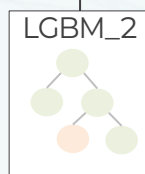
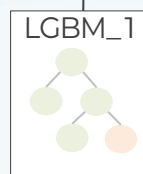
Output

- ✓ cpu_util
- ✓ memory_util
- ✓ gpu_util
- ✓ pending pods

15, 30, 60,
120분 후 예측실행 예정
리소스 정보워크로드
대기 큐 정보

Orchestration Policy Engine Multi LightGBM (Gradient Boosting Machine)

15min : 오토 스케일링, 선점
30min : 오토 스케일링, 마이그레이션, 캐싱
60min : 마이그레이션, 캐싱, 프로비저닝
120min : 마이그레이션, 프로비저닝



Task1

Node Health
Stressed
(0.72)

Task2

Autoscale
Yes
(0.68)

Task7

Storage Cache
No
(0.71)

Output

1. 노드 전체 상태
2. 오토스케일링 필요
3. 마이그레이션 필요
4. 선점 추천
5. 로드 밸런싱
6. 프로비저닝
7. 스토리지 티어링/캐싱

오케스트레이션 정책 변동

```
node_health: {
  class: "STRESSED",
  probability: 0.72,
  predicted_15min :
  "CRITICAL"
}
```

```
needs_migration: {
  class : "yes",
  probability: 0.61,
  urgency: "MEDIUM",
  priority: "LOW",
  "CRITICAL"
}
```

마이그레이션 요망

10. 정책 기반 오케스트레이션 오퍼레이터 자율 실행

개발 목표

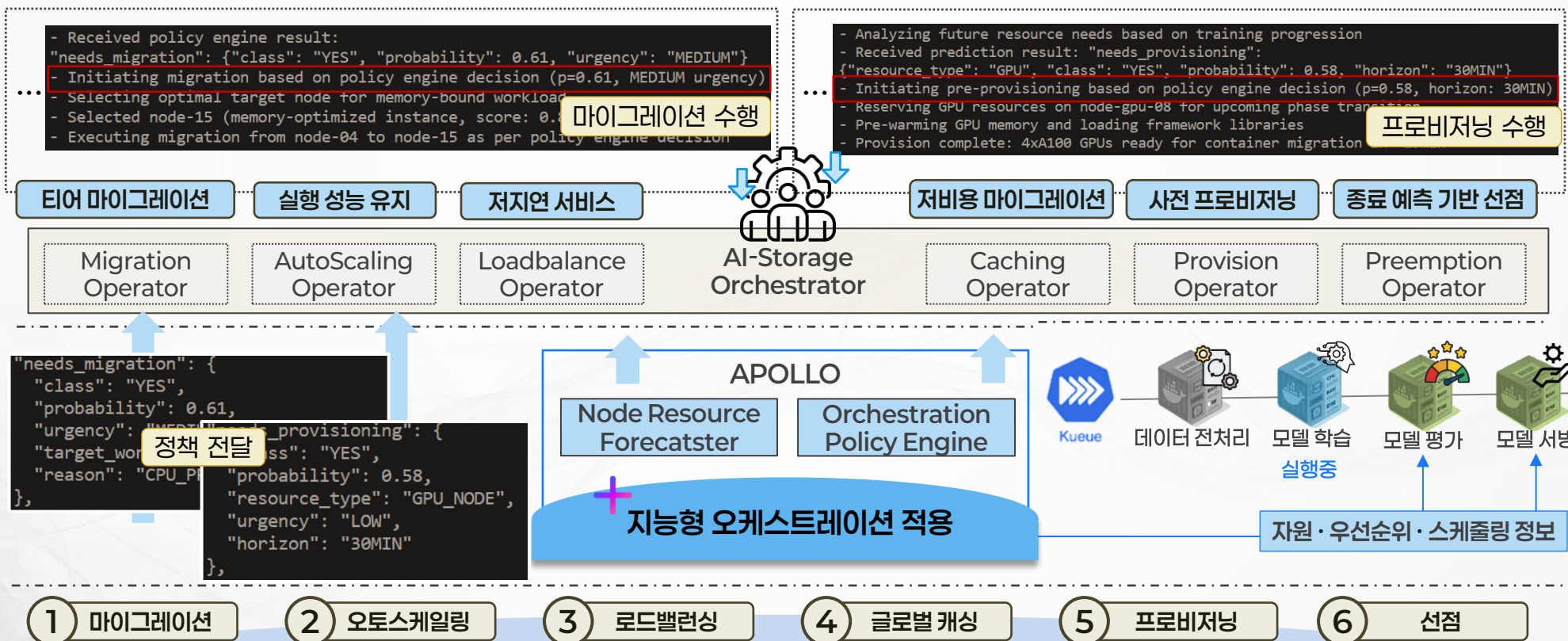
KETI 오케스트레이션 기반 기술에 **지능형 판단 모델 적용**

개발 내용

- 자원 예측 모델 결과를 반영하는 오케스트레이션 정책 연동 구조 설계
- 정책 모델에 따른 Operator 단위 (Autoscaling, Caching, Provisioning 등) 자율 실행 로직 구현

[관련 SW]

- AI-Storage Orchestrator



11. 장·단기 성능 향상을 위한 모델 재학습 구조

개발 목표

장·단기 학습 루프 기반 자기 개선형 오케스트레이션 시스템 구축

[관련 SW]

- Metric Collector

개발 내용

- 강화학습으로 스케줄링 정책을 최적화하는 실시간 정책 강화학습 피드백 ▶ 장기적 오케스트레이션 개선 루프
- 실제 운영 상황에 부합한 자원 예측을 위한 누적 데이터 기반 재학습 피드백 ▶ 단기적 자원 예측 개선 루프

장기적 모델 개선 루프

flow

- ① 환경 모니터링 (실시간)
- ② 모니터링 데이터 인코딩
- ③ 정책 실행
- ④ 보상 계산 (성능 지표 평가)
- ⑤ 정책 업데이트 (실시간 개선)

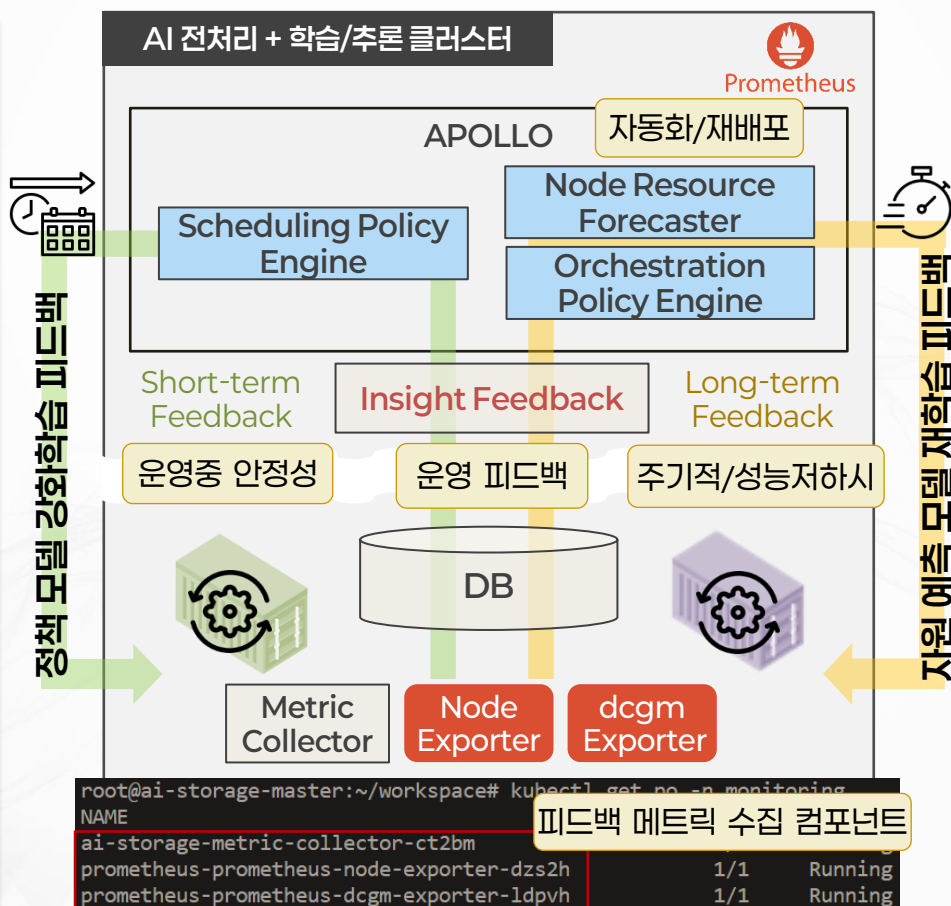
metric

```

"event_type": "node_pressure",
"status": {
  "node_id": "gpu-node-03",
  "pressure_type": "memory",
  "severity": "high",
  "threshold": 80,
  "current_value": 85,
  "predicted_time": "2023-10-27T10:30:00Z"
},
"impacted_workloads": ["sentiment-analysis", "recommendation-engine"],
"suggested_actions": {
  "migration_needed": true,
  "target_nodes": ["gpu-node-01", "gpu-node-02"],
  "urgency": "medium"
},
"sla_impact": {
  "latency_ms_increase": 150,
  "availability_risk": "low"
}
  
```

스케줄링 정책 적용 결과

성능 지표



단기적 예측 개선 루프

flow

- ① Node metrics 축적
- ② 성능 모니터링, 오차 추적
- ③ 재학습 스케줄링 (트래픽 고려)
- ④ 모델 재학습 (Sliding Window)
- ⑤ 모델 재배포 (기본: 6h 간격)

metric

```

"training_data": {
  "features": {
    "workload_type": "TFJob",
    "cpu_request": 8,
    "memory_request": 1,
    "gpu_request": 2,
    "io_pattern": "checkpoint_heavy",
    "data_size_gb": 120,
    "node_selected": "gpu-node-03",
    "storage_tier": "nvme-tier"
  },
  "environment": {
    "cluster_utilization": 0.75,
    "node_count": 5,
    "pending_pods": 3
  },
  "outcomes": {
    "job_completion_time": 1245.6,
    "avg_gpu_util": 0.85,
    "io_wait_time_percent": 0.12,
    "scheduling_accuracy": 0.94
  }
}
  
```

입력 특성

수행 결과

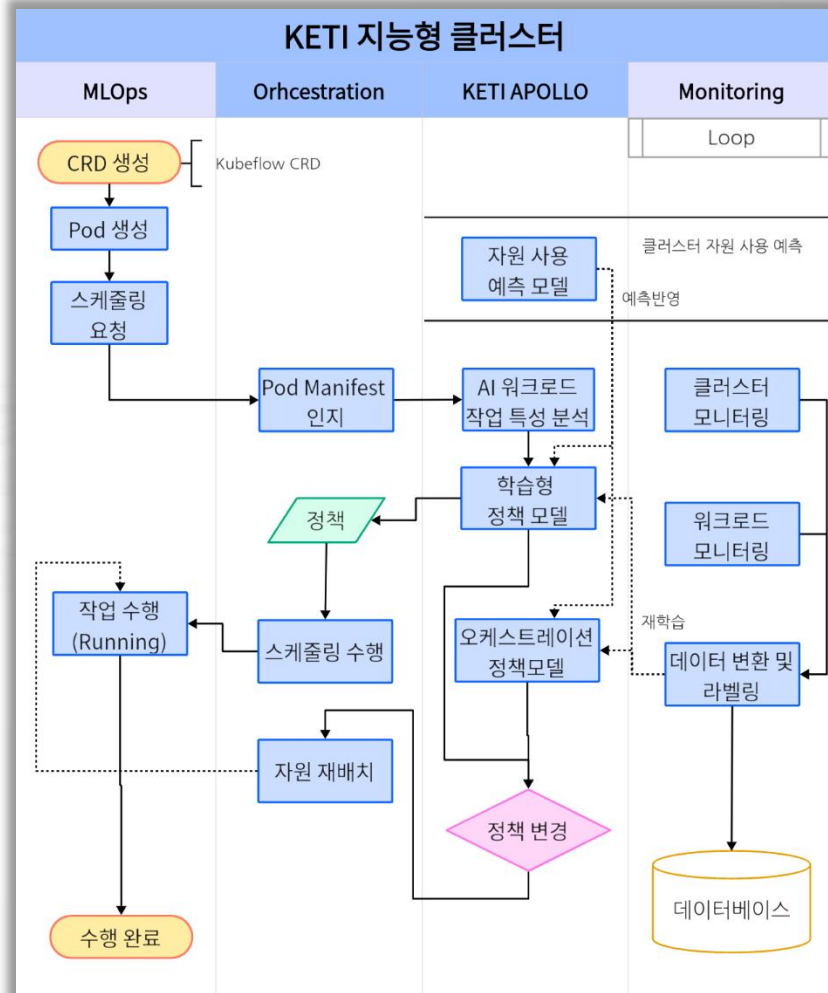
12. 클러스터 제어 패러다임 진화

KETI APOLLO로 실현하는 지능형 클러스터



KETI APOLLO

- ✓ 정책 학습·자원 예측 기반 지능형 클러스터 파이프라인 구축
- ✓ AI 워크로드 중심의 자율 제어·오케스트레이션 플랫폼 구현



CHAPTER

2

3차년도 진행 예정 사항

KETI 핵심 기술 시제품

KETI 지능형 오케스트레이션 플랫폼 : 핵심 시제품 3종 개발 상황

- 2차년도 개발 내용 ➤ 3차년도 예정 사항

I

정책 학습형 스토리지 I/O 최적화 스케줄러 모듈

- 전처리 특화 및 파이프라인 특화 스케줄링 알고리즘 7종
- 각 알고리즘별 파라미터 최적화
- 실시간 성능 피드백 기반 자가 조정 기능 구현

[시제품 대비 구현 상황]

시제품 대비 70% 구현

시제품 100% 구현

3차년도 상반기 워크샵
이후 실장비 기반 통합 예상

II

자원 예측 기반 클러스터 오케스트레이션 모듈

- 검증용 정책 변경 이벤트로 동작하는 오케스트레이션 개발
- 오케스트레이션 모듈 - MLOps 파이프라인 결합 구조 개발
- 오케스트레이션 시스템 동작 검증 및 안정화

시제품 대비 80% 구현

III

학습형 정책 결정 모델 & 자원 사용량 예측 모델 (APOLLO)

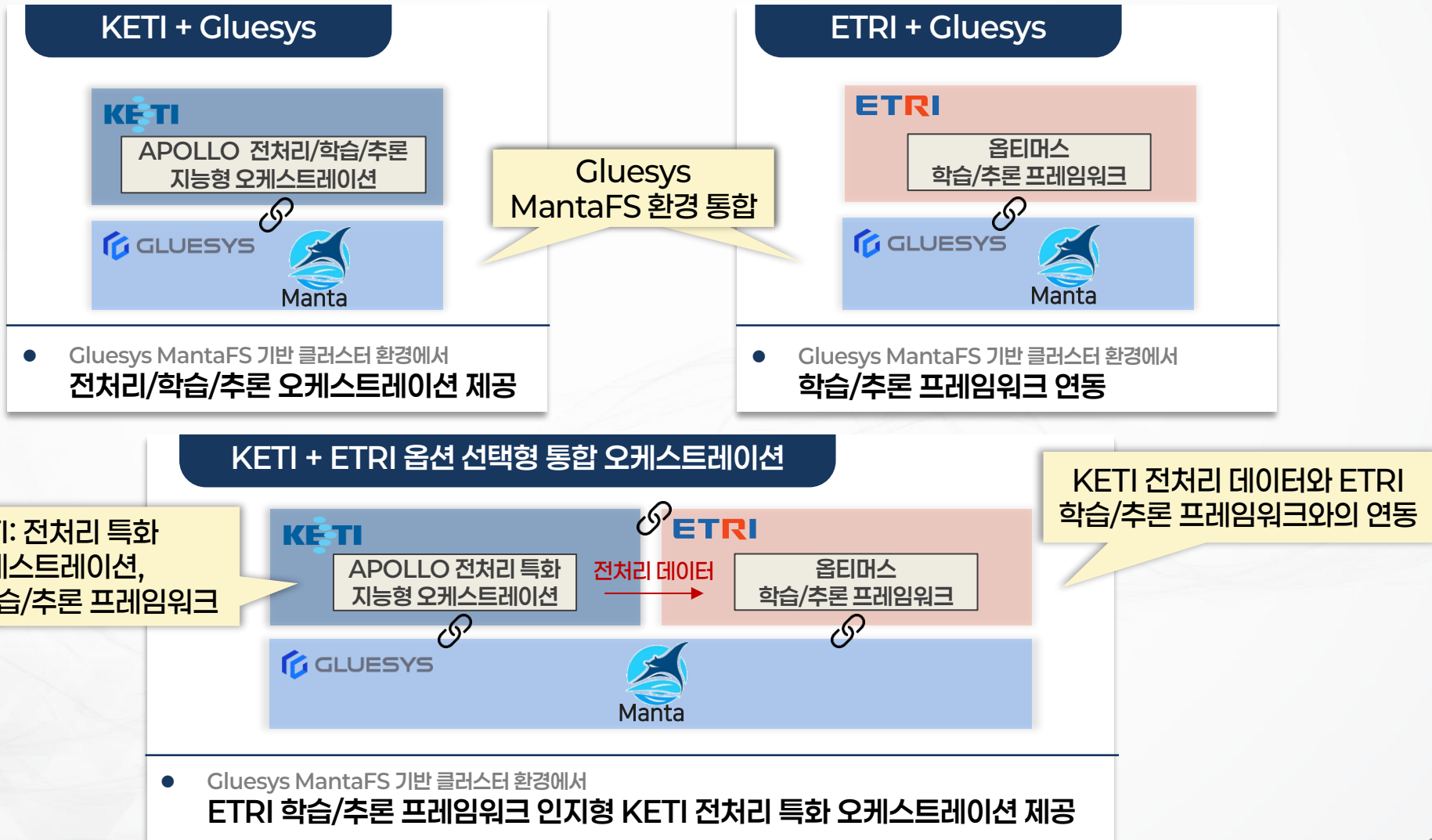
- 목적 함수 기반 정책 결정 및 기본 메트릭 자원 사용량 예측 모델
- 정책 결정 모델 및 자원 사용량 예측 모델 재학습 루프 개발
- 스토리지 특성 반영, 자원 사용량 예측 모델 안정화

시제품 대비 50% 구현



3차년도 연계 구조

MantaFS 기반 기관별 예상 연계 구조



CHAPTER

3

**2차년도 산출물
및
향후 일정**

정량 평가 항목

#7 AI 학습 컨테이너 스케줄링 노드 선정 시간

- 당해 목표 : — (3차년도 목표 2초 이내) 3차년도 목표치 대비 달성도 자체 확인
- 평가 방법 : 자체 검증 진행
- 연관 개발 기술 : 학습형 정책 적용 스케줄러
- 측정 방법 : 30개의 job을 배포하는데 소요되는 시간 측정
- ✓ 스케줄러가 Queue에서 작업을 꺼낸 시점부터 etcd에 기록되는 시점의 간격을 측정

➤ 검증 결과

```
09:45:00.000 Scheduler starting batch processing of 30 jobs
09:45:00.083 Job 1: shceduled, recording to etcd
09:45:00.167 Job 2: shceduled, recording to etcd
...
09:45:01.838 Jobs 30: shceduled, recording to etcd
09:45:02.503 All 30 jobs: node selection and etcd recording completed
```

```
=== Scheduling Latency Metrics ===
Total Jobs Measured: 30
First Job Scheduling Start: 09:45:00.000
Last Job Decision Completed: 09:45:02.503
Total Scheduling Time: 2.503 second
Jobs Per Second: 11.98
Queue Processing Rate: 83.5 ms per job
Node Evaluation Speed: 64 nodes in
=====
```

30개 job 2.503 소요

3차년도 목표치 대비 80% 달성

#8 AI 학습 컨테이너 마이그레이션 CPU 절감율

- 당해 목표 : 30% 이내 11월 말 진행 예정
- 평가 방법 : 공인 시험 예정
- 연관 개발 기술 : 자원 예측형 오케스트레이터
- 측정 방법 : 기본 구현 마이그레이션과 KETI 오케스트레이터의 마이그레이션 비교
- ✓ K8s는 기본 마이그레이션 기능이 없기 때문에 Pod 삭제 후 재 스케줄링 방식으로 기본 마이그레이션을 구현

➤ 검증 결과

```
=== CPU Utilization Comparison Results ===
Test Duration: 2 hours per migration type
Workload: ResNet50 training (ImageNet dataset)

--- Post-Migration CPU Utilization ---
Basic K8s Migration: 76.8% CPU utilization
KETI Orchestrator: 55.2% CPU utilization

--- Comparison Metrics ---
Absolute Reduction: 21.6 percentage points
Relative CPU Reduction: 28.2%
Target Achievement: 94.0% (target: 30% reduction)
=====
```

CPU 절감률 28.2%

2차년도 목표치 달성

정량 목표 달성 현황

☞ 목표 : 기술문서 2건, 국내 논문 -건, S/W 등록 2건, 특허 출원 2건

평가 항목	목표	성과 지표 달성 현황 : 2025.11.06 기준	완료 및 예정 날짜
기술문서	2건	기술문서 2건 작성 완료 - AI 고효율 스토리지_고효율 스토리지 기반 AI 워크로드 오케스트레이션 시스템 아키텍처 설계서 - AI 고효율 스토리지_인스토리지 기반 AI 학습 데이터 전처리 기술 설계서	25' 8월 12일
논문	국내 - 건	국내 학술대회 1건 완료 연산 스토리지 기반 전처리 오프로딩을 적용한 AI 컨테이너 오케스트레이션 아키텍처 설계에 관한 연구	25' 09월 24일
SW 등록	2건	SW 2건 등록 완료 - 연산 스토리지 기반 AI 전처리 워크로드 오프로딩 모듈 - AI 고효율 병렬 스토리지 기반 워크로드 고속 수행을 위한 오케스트레이션 모듈	25' 10월 1일
특허	2건	국내 출원 3건 진행중 발명명세서 전달완료 - AI 플랫폼 환경에서 호스트 CPU 절감을 위한 전처리 컨테이너 CSD 오프로딩 적용 방법 - AI 학습 컨테이너 환경에서 자원 사용량 예측 모델 기반의 지속적 스케줄링을 지원하는 방법 - AI 플랫폼 환경의 연산 스토리지 내 학습 모델 기반 메타데이터 생성 및 데이터 큐레이션 기술	25' 11월 14일

향후 일정



실증 환경 기반 통합 연동 준비

추진 일정

9	10	11	12	1	2	3	4	5	6
고효율 AI 스토리지 환경 기반 AI 학습 컨테이너 고속 배치 스케줄링 구조 개발				고효율 AI 스토리지 환경 기반 AI 학습 컨테이너 고속 배치 스케줄링 구조 통합 연동					
고효율 AI 스토리지 환경 기반 AI 학습 컨테이너 자원 사용량 및 정책 예측 기술 개발						고효율 AI 스토리지 환경 기반 AI 학습 컨테이너 자원 사용량 및 정책 예측 기술 통합 연동			
고효율 AI 스토리지 환경 기반 AI 학습 컨테이너를 위한 오케스트레이션 구조 개발							고효율 AI 스토리지 환경 기반 AI 학습 컨테이너를 위한 오케스트레이션 구조 통합 연동		

정량항목 공인시험

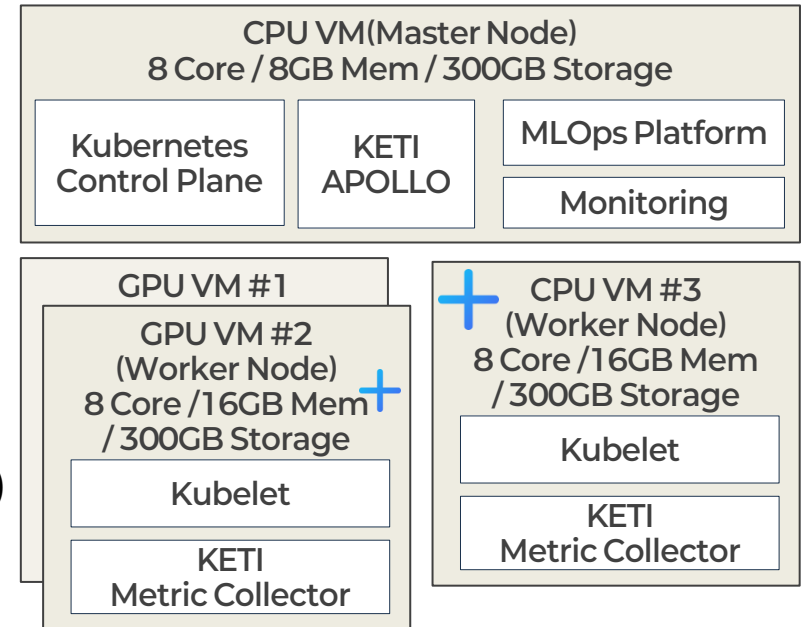
2차년도 연차 보고서

테스트 환경
사전 통합

논의사항

1. 3차년도 실장비 기반 통합에 앞선 VM 기반 테스트 환경 구축 관련 검토 사항

- 클루시스 제공 가상화 환경
 - ✓ CPU VM: 1대 (테스트 환경 운영용, Master Node)
 - ✓ GPU VM: 2대 (A6000 vGPU, 8core / 8GB Ram / 300GB Disk)
 - ✓ (추가요청) + 8GB Ram
 - ✓ (추가요청) CPU VM: 1대(CPU Worker Node)
 - ✓ 네트워크: 1G 접속망 + IB 내부 공유망
 - ✓ 스토리지: 클루시스 공유 볼륨 연결 제공
- 설치 환경 요구사항
 - ✓ OS 버전 : Ubuntu 22.04 LTS^(long-term support 선호)
 - ✓ 원격 접속을 위한 외부 시작 IP 공유 예정
 - ✓ VM 메모리 16GB
 - ✓ 설치 예정 : Kubernetes(v1.28+), Kubeflow(v1.8)
Prometheus, Grafana, ArgoCD, Kueue



2. 연동 인터페이스 협의

- 스토리지 계층 정보 인터페이스 : Manata 클러스터 계층별 성능 지표 및 상태 정보 수집 API
- 데이터 지역성 스케줄러를 위한 데이터 볼륨 물리적 위치 정보 공유
- 동적 티어링 환경에서 워크로드 성능에 미치는 영향 피드백 체계 필요
- 데이터가 계층 간 이동할 때 스케줄링 정책에 반영 할 수 있는 이벤트 알림 체계 필요



감사합니다

안재훈 책임 (KETI)

TEL 010-5558-6857

EMAIL corehun@keti.re.kr
